

Optimized Ride Matching Engine

A C++ console-based ride-sharing simulation demonstrating real-world software engineering concepts — from polymorphic fare strategies to a multi-factor smart driver selection algorithm. This system models a complete dispatch pipeline: rider requests, driver availability tracking, dynamic pricing, ride logging, and an analytics dashboard with fairness metrics.

By: Shanzay Abid Malik

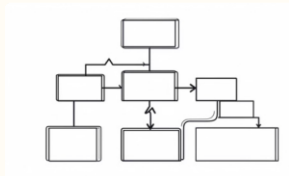
C++ PROJECT REPORT

OOP DESIGN

ALGORITHM DESIGN

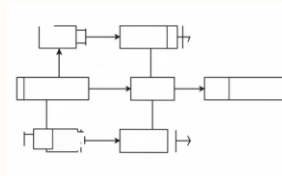
Core Architecture & Class Design

The system is built on a clean object-oriented hierarchy using inheritance, abstraction, and encapsulation. Every entity 4 riders, drivers, locations, and ride records 4 is modeled as a first-class object with well-defined responsibilities.



User Inheritance Hierarchy

User is an abstract base class defining `userID`, `name`, and `Location`. `Rider` and `Driver` extend it, each overriding `displayProfile()` for polymorphic behavior.



Driver State Machine

Drivers track `isAvailable`, `rideCount`, `fatigueLevel` (0-100%), and `lastRideTime`. A 45-second auto-reactivation timer simulates realistic trip completion cycles.



RideRecord & Logging

Every completed ride is captured in a `RideRecord` struct 4 storing passenger/driver info, distance, fare, pricing mode, algorithm used, and a cross-platform safe timestamp. Records persist to `dispatch_manifest_logs.txt`.

Key OOP Concepts Applied

- Abstract base classes with pure virtual functions
- Runtime polymorphism via virtual destructors
- Encapsulation of driver state and location data
- RAII-style resource management in `RideMatchingEngine`

Location & Distance Model

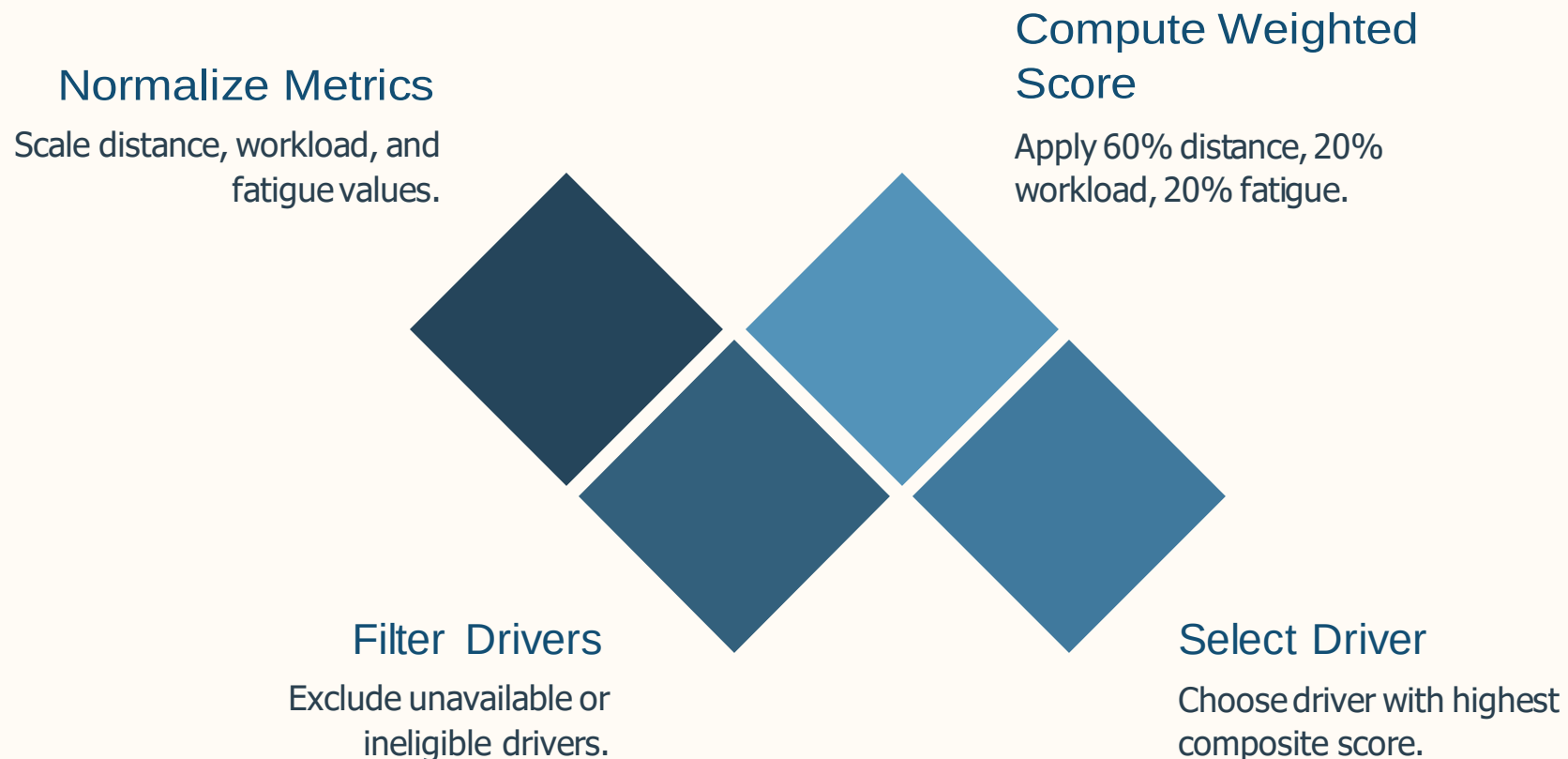
The `Location` struct uses 2D Euclidean coordinates. Distance is computed via:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

After each ride, the driver's location updates to the passenger's destination 4 enabling spatially accurate subsequent matching.

Smart Driver Selection Algorithm

The system's centerpiece is a **weighted multi-factor scoring algorithm** that selects the optimal driver based on three normalized metrics. Unlike a naive nearest-driver approach, this algorithm promotes fairness, reduces burnout, and improves fleet utilization.



Each candidate driver receives a composite score between 0 and 1. Drivers with fewer than 3 rides today receive a **+0.05 bonus**, encouraging equitable distribution from the start of a shift.

Scoring Formula

$$\text{Score} = (d_{\text{norm}} \times 0.6) + (w_{\text{norm}} \times 0.2) + (f_{\text{norm}} \times 0.2)$$

Where each normalized component ranges from 0 (worst) to 1 (best). A driver who is close, lightly loaded, and rested will score near 1.0.

Why 60 / 20 / 20?

- ♦ **60% Distance**— Proximity remains the dominant UX factor
- ♦ **20% Workload**— Prevents over-assigning to popular drivers
- ♦ **20% Fatigue**— Protects driver well-being and safety

The algorithm is toggleable at runtime, allowing direct A/B comparison against a basic nearest-driver fallback.

Fare Strategies, Analytics & System Capabilities

Beyond matching, the engine implements dynamic pricing, a full analytics dashboard, and a ride history log — making it a complete simulation of production ride-sharing infrastructure.

Standard Pricing

Base fare of **50 PKR** plus **12 PKR/km**. Applied during normal demand periods.

Surge Pricing

Configurable multiplier (e.g., 1.5×, 2.0×) applied to the base formula during high-demand events. Switchable at runtime via the control panel.

Analytics Dashboard

Reports total rides, distance, revenue, per-driver performance, and a **Fairness Score** (0–100%) measuring ride distribution balance across the fleet.

Auto Reactivation

Drivers become unavailable during a ride and automatically reactivate after **45 seconds**, simulating realistic trip completion and re-entry into the available pool.

Algorithm Comparison Output

The system can display a side-by-side comparison of Smart vs. Uber-style (nearest-driver) performance, including fairness metrics and a recommendation engine that suggests switching algorithms when imbalance is detected.

 **Fairness Score Thresholds:** ≥80% = Excellent · ≥50% = Moderate · <50% = Poor — with automatic recommendations to switch algorithms.

System Highlights at a Glance

- ◆ Polymorphic fare strategy pattern (Strategy
- ◆ Pattern) Smart weighted scoring with
- ◆ normalization
- ◆ Driver fatigue and workload tracking
- ◆ Cross-platform safe timestamp handling
- ◆ Persistent ride logging to file
- ◆ Interactive 10-option control panel
- ◆ Exception-safe menu loop with input validation